BIG DATA TESTING METHODOLOGY

Big data technologies present challenges in Quality Assurance (QA) that are unique to this area and cannot be addressed by regular testers. About 21% of big data developers, surveyed worldwide, have stated that data quality is the biggest problem affecting big data systems. Thus there is a need for specialized testers in the field of big data testing.

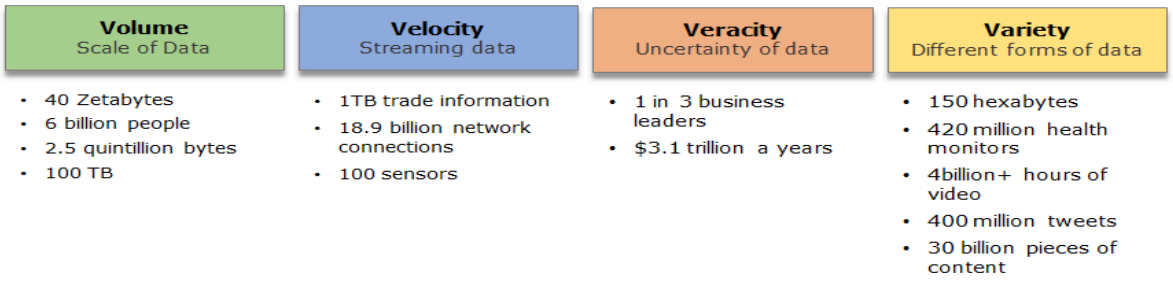
Need for Big Data testing:

- Big data technologies have immense power over information. However, if proper checks are not in place, it can lead to a lot of **customer concerns**.
- Using big data is advantageous in getting insights. However, the information has value only if it is obtained at the correct time. Therefore, **performance testing** is important.

Given the huge volume of data, there are challenges in testing BD systems:

- Live integration of information: since data are fetched from different sources, it is necessary to facilitate their live integration. Data integrated from multiple sources may cause some errors. Need for end-to-end testing of data sources and integrators.
- Instant data collection solutions and deployment: the application must be thoroughly tested and certified for live deployment.

AN OVERVIEW OF BD SYSTEMS



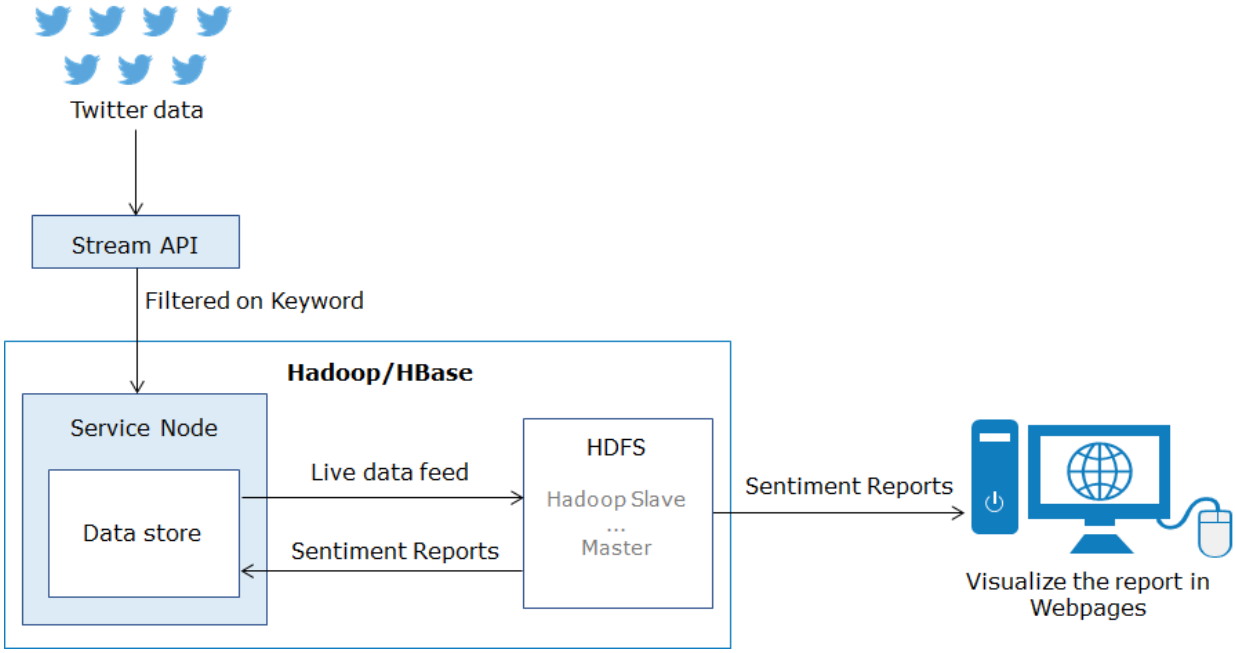
2 popular system solutions to harness the power of BD:

- NoSQL: a non-relational, distributed, open-source and horizontally scalable database, schema-free, flexible, supported by distributed architectures, omit joins and transactions to enable scalability. Types: key-value store, column family, document based, graph store.
- Hadoop: an open source framework, provides reliable and scalable distributed computing. While designed to be user-friendly, Hadoop in its “raw” state still needs considerable specialist knowledge to set up and run.

Forms of BD implementations

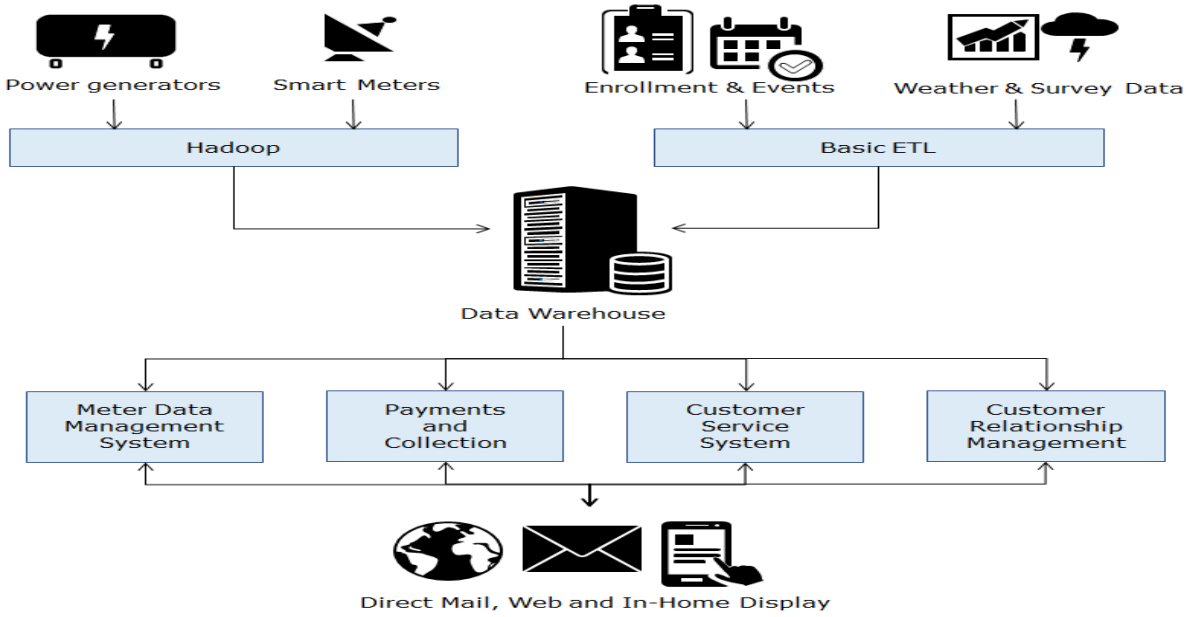
1. Standalone Hadoop setup for data processing

Only Hadoop is used to capture, store and process data.



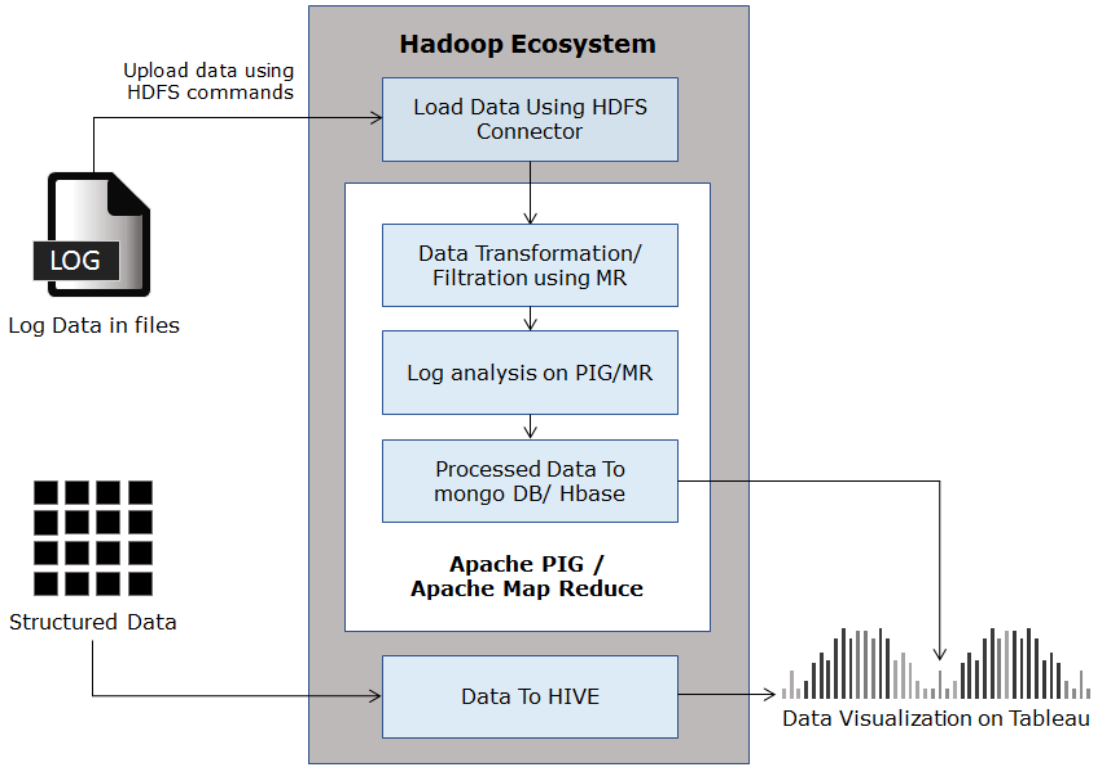
2. EDW(enterprise data warehouse) integrated with Hadoop for ETL processing

Part of the data is captured into Hadoop from some of the sources, while the rest is traditionally captured and stored directly in a data warehouse from other sources.

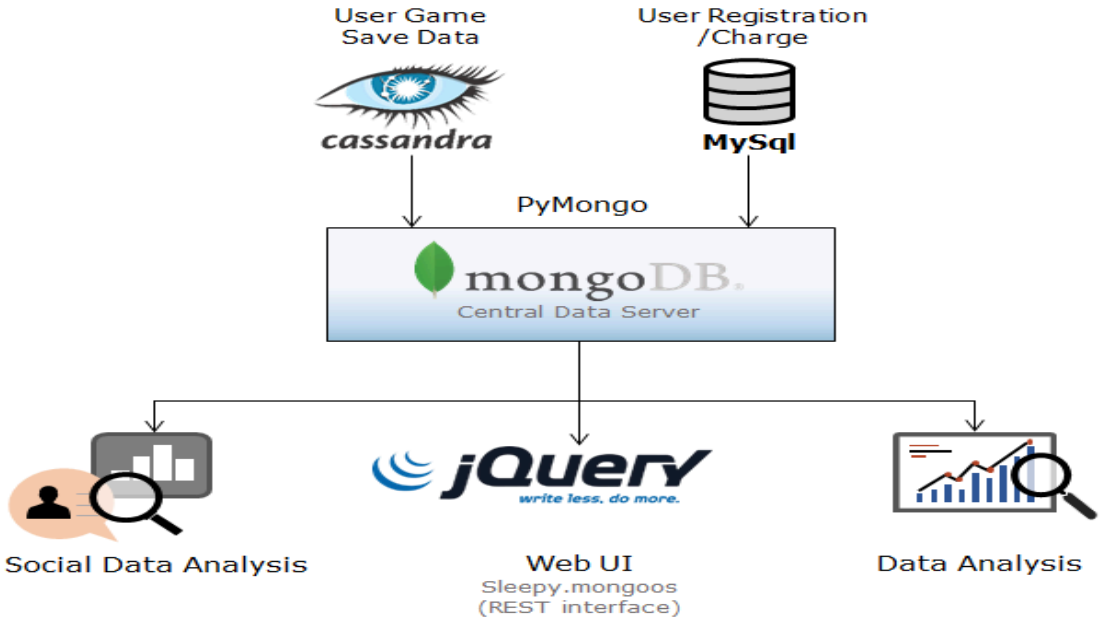


3. BI reporting tools setup on big data eco-system

A reporting tool connects to the big data store to fetch data and create reports.



4. Data stored in NoSQL database



Applications

Manufacturing

Cyber physical models: Physical machinery monitored/controlled by computer algorithms.
Data collection: real-time sensor data and historical data.
Usage: Failure prediction, Prognostics and Health Management (PHM).
Alstom uses BD to scan sensor data from locomotives every 45-90 s for maintenance optimization.

Banking and financial services

Fraud detection & prevention: Identifies and prevents potential fraud.
Customer segmentation: Analyzes spending habits to recommend personalized products.

Retail

Consumer sentiment analysis: Walmart analyzes over 1 million transactions per hour to predict buying patterns and implement PoS offers.

Medicine

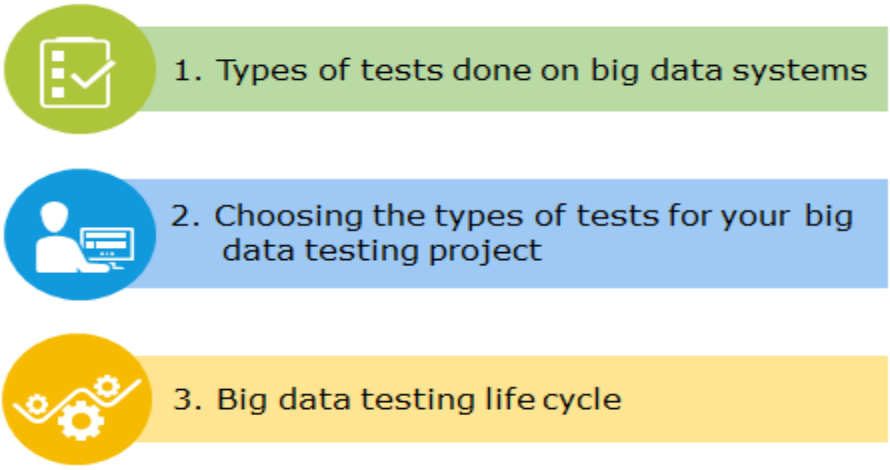
Big-data Clinical Trial (BCT): Analytical solution for diagnostics and clinical drug tests.
Kenneth Cukier's TED Talk: BCT discovered new malignancy parameters in breast specimens, highlighting the collaboration between physicians and data scientists.

Difference between data warehouse testing and BD testing

Based on volume, both are used for gathering and analyzing large volumes of data. However, while a data warehouse might process data in the order of gigabytes, in BD applications, the data volume can easily scale to petabytes.

	Data warehouse testing (DWT)	Big data testing (BDT)
Type of data	DWT deals with mostly structured data only.	BDT has no such constrains and deals with structured, semi-structured and unstructured data.
Type of source	DWT mostly involves testing batch processing of data.	BDT mostly involves testing live streaming data.
Environment and infrastructure	DW systems are based on RDBMS systems. Hence the tester needs to know how to write SQL queries.	BD systems are based on file systems. Hence the tester needs to know how to work with commands and query languages specific to big data systems such as Hadoop shell commands, Hive QL etc.
Tools availability	There are a lot of DWT tools available in the market such as Informatica DV as all automation is based on SQL queries.	There are not many tools available for big data testing as the data querying and retrieval techniques are unique to each BD system. If automated tests need to be created, they need to be done using the BD systems' inbuilt features such as MapReduce for Hadoop.

HOW BD SYSTEMS ARE TESTED



TYPES OF TESTING

1.1 Functional Testing

- Data Ingestion / acquisition
- Data Quality Analysis
- Data Processing / Data Mining Testing
- Reports and Visualization Testing

1.2 Non functional Testing

- Infrastructure Testing
- Big Data Appliance Setup Testing
- Performance Testing
- Data Security Testing
- Failover Recovery Testing

Functional testing

Importance

Data scientists spend 60% of their time cleansing data.
Complexities in data gathering can lead to errors like bad headers, incorrect data types, and unwanted character encoding.

Requirements

- Movement of data
- Processing of data

Types

1. Data ingestion testing
    - External systems data
    - Real-time data integration
    - History data migration
2. Data quality analysis
    - To ensure the quality of stored data.
    - Ensuring no duplicate information.
3. Data processing testing / Data mining testing
    - To process the data stored and bring them to a usable/understandable/actionable format, various tools like MapReduce, Hive, Pig, Spark, etc. are used. To further enhance this analysis process, various algorithms are used and data mining/analytical processes are performed.
    - Tools: MapReduce, Hive, Pig, Spark.
    - Example: Correct extraction of preferences for discounts.
4. Reports and Visualization Testing
    - Ensuring accurate visual representation.
    - Example: Decline in customer footfall shown with a red, downward arrow.

If the source data is not in the format that can be used in the BD platform, then one or more of the following actions need to be performed:

- **Cleansing** the required data (Eg. Extract only the first sign-in and last sign-out transaction records from the log)
- **Filtering** unwanted data (Eg. Delete all the transaction records from the log except the first sign-in and last sign-out)
- **Transforming** the data into meaningful data (Eg. Convert the transaction record string into Julian date format and 24-hour timestamp format)

Mode of uploaded data, based on the source:

- Batch upload (Eg. financial transaction data)
- Real time data upload (Eg: Sensor data, social media data like posts, tweets etc.)

In order to verify the proper functioning of all the above steps, perform data ingestion testing by validating whether:

- the right data is getting extracted from source systems and loaded into the correct HDFS location
- the transformation rules are correctly applied
- data integrity is maintained
- there is any missed/additional data by comparing source data with data ingested

Types of data ingestion testing

Depending on the types of data sources and direction of the data flow.

Data Acquisition testing

Done to achieve accurate/valid and relevant data. Data captured from various sources are stored in BD storage devices. This is the most vital phase in the whole data processing process. Many sources do not have proper data sanitization done at the entry point of the data. There remains a lot of noise in the data which needs to be removed.

E.g. Continuously generated data from a heart rate monitoring device needs to be filtered for only relevant data (Eg: for work-out durations) before being uploaded in the HDFS system.

Data Integration testing

When the data from the BD storage devices and the data from the traditional data warehouses are integrated, the data is either moved from the RDBMS data warehouse to BD storage or vise-a-versa. This integration/migration needs to be tested.

E.g. If there is a need to integrate customer data present in the social networking sites and their shopping behavior in supermarkets, then there needs to be a check that only public information is stored and not the private profile information.

Data Migration testing

Historical data might be present in the enterprise data warehouse. This data might need to be migrated to the BD storage for integrated processing. need to verify that all the data from the data warehouse is copied as it is.

E.g. Confirming successful migration by comparing the source tables in the data warehouse and destination records in the HDFS system for contents and size.

When a text file has been migrated from a Unix system to an HDFS system, confirm the following details to validate the migration :

- Check if the file **is present** in the HDFS
- Check if the file **size matches** in both the systems
- Check for **content match** by comparing source file against the target

These data comparison tests are normally performed by downloading from HDFS and comparing against local Unix file.

```
diff <(hdfs dfs -cat /source/abc.log) <(cat /destination_downloaded/xyz.log)
//store the result and give appropriate message
diff <(hadoop fs -cat $2) <(cat $1) > comparefileresult.txt
chmod 777 result.txt
count=$(wc -l < comparefileresult.txt)
if [ $count -ne 0 ]
then
	echo "DATA mis-Matched & Mismatch Data are in result.txt"
else
	echo "DATA matches "
fi
```

Data ingestion testing is one of the most significant aspects of BD testing as it is conducted at the beginning of the entire data flow path. It is uniquely placed to find defects early in the system, which can stop multiple secondary defects as the data flow proceeds.

Challenges involved in data ingestion testing

- Complexity of transformation rules applied between source and destination
- Enormous quantity and variety of data
- Integrity of data in heterogeneous data stores
- Multiple interfaces
- On the fly data connectors
- Performance issues

Key skill needed for data ingestion testing is knowledge of technical characteristics of different file formats that are used by the source systems and the big data systems.

Data quality analysis

Standard quality analysis checks :

Metadata analysis

Performed on the metadata/structure of the data stored.
Example: whether the Hive table structure matches the designed structure/schema, and the data is stored in the required file format.

Pattern analysis

performed on certain fields which should have a specific pattern of data.
Example: email should have a "@" and ".com" or ".id", date should be stored in "dd/mm/yyyy" or "dd-mmm-yy" only.

Statistical analysis

Check if the values of certain fields are within their statistically acceptable limits.
Example: Check if the value is within acceptable range, the average, maximum or minimum value of a numeric column is acceptable, if a particular field contains unique values.

Relationship analysis

Check the cardinality of values i.e. whether two values, in the same or different table/file, have 1:1, 1:m or 0:m relationship.
Example: each entry in the manager table must have a 1:m relationship with unique entries in the employee table.

Business rule validation

Check if the data stored is confirming to any business/domain specific requirement.
Eg: account type should be "current" or "FD", the HRA should be less than 30% of basic salary.

<u>Challenges while testing data quality:</u>

- Huge volume of data to be analyzed
- Lack of constraints in some of the BD stores
- Partitions in BD stores
- Traditional SQL querying not being useful
- Working with NoSQL databases which use proprietary query languages
- Complexities involved in creating non-GUI test automation scripts
- Absolute need for complete or semi-automated testing

Data processing / mining testing

Data need to be processed in order to derive meaningful insights from it.
Data can be extracted using Spark, HiveQL, MapReduce, or any query language specific to NoSQL DB.
Validate correctness of values.

Resorts and visualization testing

Can be classified into 2 broad test areas:

Testing the UI

Refers to the look and feel of the dashboard page in terms of usability.

Parameters to be validated -

- Dashboard's page name and labels
- Layout of sub reports which are present in the dashboard page
- Prompts which are displayed in the dashboard page
- Values that need to be displayed in the prompts
- Columns that need to be present in the report.
- The correctness of "Sum Total" of the values displayed in the report.
- Format of the values displayed in the columns of the report
    - E.g : The fields indicating a ‘Revenue’ must have ‘$’ sign preceding the value, comma separator for every thousand’s place value,2 places after the decimal point.
- The availability of column drill-down features and the correctness of their paths.
- If a graphical representation of the data is present, then the correctness of the graphical representation (pie chart, bar chart etc.) of the data.
- The availability of filter conditions which have been used, displayed below or above the report.
- If the filters are common for the graphs and tables, then the similarity in values of both the reports upon changing the filter conditions.
- Availability of user actions like ‘Refresh’, ‘Print’, ‘Download’ options below the report.
- The formats available for downloading reports.

Validating the data in the report

Done by executing queries against the data stores and comparing their results with what is there on the report.

Guidelines -

- Go through the requirements (documents, templates etc.) and discuss with the developers.
- Ascertain if any changes or enhancements are done to the data stores and how it will be affecting the front end, the reports and particular columns.
- Create queries as per the requirement document and data stores' structure/schema to extract the required data from sources.
- Compare the extracted data against the data in the report.

CHOOSING THE REQUIRED TYPES OF TESTING FOR THE BD PROJECT

4 BD testing assignments:

1. Migration testing

Done whenever data is migrated from one storage type to another. Example, EDW to Hadoop.

2. End-to-end testing

Creating new data lakes and data hubs leads to replacing existing data warehouse systems to store data in different zones for easy retrieval. Such implementations require all the aspects of the big data system to be validated (end-to-end). Steps:

   a. Data ingestion testing: ensure correctness of data from various sources.
   b. Data quality analysis: validate data integrity and business rules post-ingestion.
   c. Data processing testing: verify data filtering and transformation based on business logic.
   d. Data visualization and reports testing: validate reports, dashboards, and mobile outputs against business requirements.

3. Report testing

Focus on data visualization and reporting.

4. Data archival testing

Seen in BD stores used for storing data for audit and compliance purposes. In such systems, the data is not processed and is stored as it is to ensure that it can be retrieved easily. The validation approach involves source-to-target comparison, where data, from source databases, is validated against target data in big data stores.

BIG DATA TESTING LIFE CYCLE

1. Requirements Analysis

Look for information in order to understand:

- Various use-cases in-scope for testing
- Various technology components (Hadoop/Map Reduce, Big Data appliance, BD connectors etc.) involved in processing Big data
- Various data storage (HDFS, NoSQL DBs, Hive flat files etc.) requirements
- Data sources and source transaction information
- Data integration requirements like integrating social and enterprise data
- Various data types (structured, unstructured) in-scope
- Detailed data extraction and processing requirements
- Testing types in scope

In case report testing is involved, gather information like:

- Detailed report data requirements
- Report layout and format requirements
- Various tools (Pentaho, Microstrategy etc.) used for generating reports and dashboards
- Various mobile devices in-scope for validating the reports

2. Test Planning

Create a detailed plan of the testing with the following tasks:

- Identify key components involved in testing
- Identify the utilities and tools required for testing
- Document detailed test strategy and approach for each testing type in scope
- Document detailed test cases based on the data extraction and processing requirements
- Identify test environment requirements, setup and management process
- Collaborate with infrastructure team to set up test environment with optimal number of data nodes required for testing the data, configure the cluster as per testing need
- Understand test data needs and identify or prepare test data

In case report testing is involved, below information is gathered.

- Identify types of reports in testing scope
- Document detailed test strategy and approach for validating reports and dashboards across platforms (PC, Tablets, Phones etc.)
- Document detailed test cases based on the report requirements

- Collaborate with infrastructure team to set up reporting environment

3. Test preparation

Create the test environment and check for its testing readiness by:

- Performing environment readiness check to ensure multi-node cluster is up and running and verify the configuration
- Preparing validation scripts to validate the data present in HDFS, NoSQL or Hive tables
- Identifying test data by sub-setting production data or preparing data as per test cases

Additional step in case of report testing:

- Preparing validation scripts to validate the data present in reports and dashboards

4. Execution

- Perform smoke test to check if required files are extracted and loaded into processing area
- Execute test cases documented and report test results
- Perform data quality check on the source data post extraction
- Identify discrepancies and log defects
- Perform defect retesting and regression testing

In case report testing is involved,

- Perform smoke test to check if reporting tool is connecting to right sources and able to run the reports without any errors
- Execute test cases documented and report test results
- Validate the reports data and dashboard data against source data
- Identify discrepancies in report and dashboard data and log defects

COMMON ERRORS IN BD SYSTEMS

Missing data

Data that does not get into the target data store in its entirety.
Causes:

- Invalid or incorrect lookup table in the transformation logic.
- Bad data from the source databases (need cleansing) in valid joins.

Truncation of data

Data being corrupted by truncation of the data field.
E.g. “New Mexico City” truncated to “New Mexico C” .
Causes:

- Invalid field lengths on the target database.
- Transformation logic not considering field lengths from source.

Data type mismatch

Data types not correctly set up in the target databases.
E.g. The source data field was required to be a date. However, when initially configured, it was set up as a VarChar.
Cause: Source and target data field not configured properly.

Null translation

Null source value not being correctly transformed when placed in the target data store..
Cause: The developer did not include or incorrectly included the null translation logic.

Wrong translation

Refers to instances where the field should have been null but is populated with a non-null value or a wrong value.
E.g. The target field should only be populated when the source field contains certain values, otherwise should be set to null.
Cause: Incorrect logic in the translation.

Misplaced data

This occurs when source data is moved to an incorrect target data field.
Cause: Inaccurate mapping of the source to the target field.

Extra records

Records that are not supposed to be loaded are loaded to target data stores.
Cause: Filter logic was not implemented.

Not enough records

Records which should be there are not there.
E.g. If a case was in a certain state, it should’ve ETL’d over to the data warehouse, not the data mart.
Cause: The development team had a filter which was not required.

Transformation logic error / holes

There are "holes" in the logic or realizing that some requirements/logics are not clear.
E.g. Product names on a case should be separated by a comma and then a space but target field only has it separated by a comma.
Cause: The developer did not consider special cases.

Simple / Small errors

Capitalization, spacing, etc..
E.g. Duplicate records in the sales report were doubling up several sales transactions which skewed the report significantly.

Undocumented requirement

Finding requirements that are understood but not documented anywhere.

Duplicate record

Same data repeated in more than 1 record.
Cause: Appropriate logic to fetch only unique records not in place.

Numeric field precision

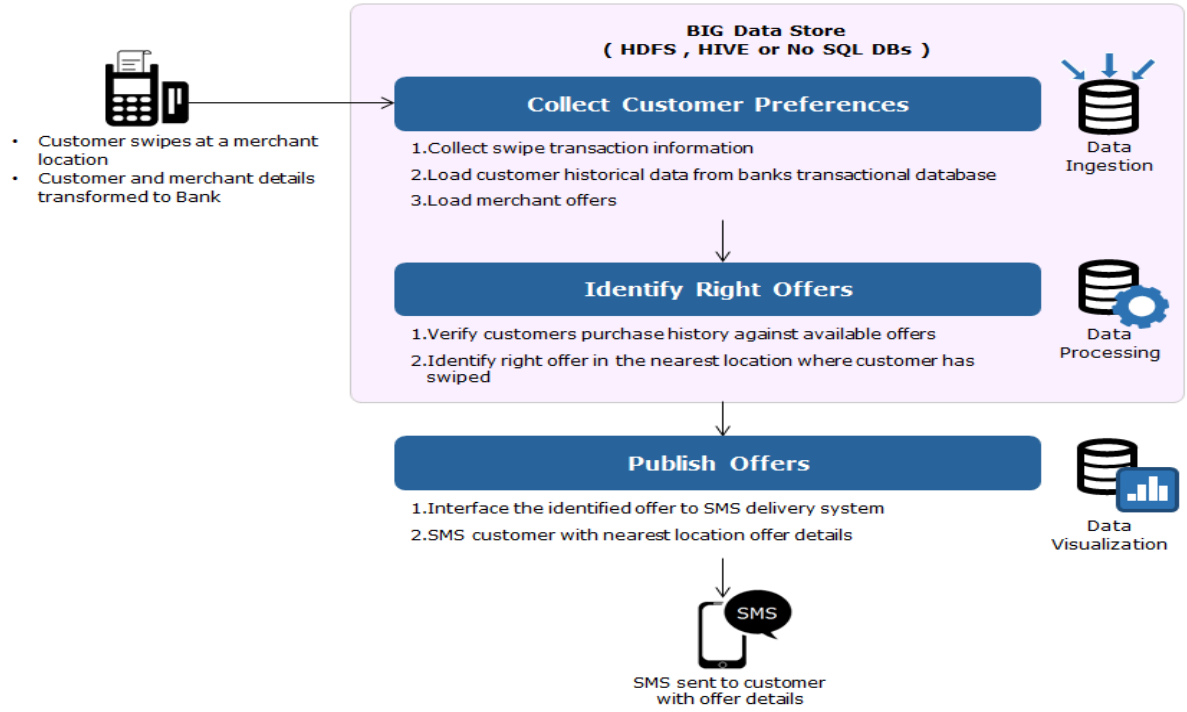
Numbers that are not formatted to correct decimal points or rounded as per specification.
E.g. The sales data did not contain the correct precision and all sales were being rounded to the whole dollar.
Cause: Program implemented with incorrect precision.

Rejected rows

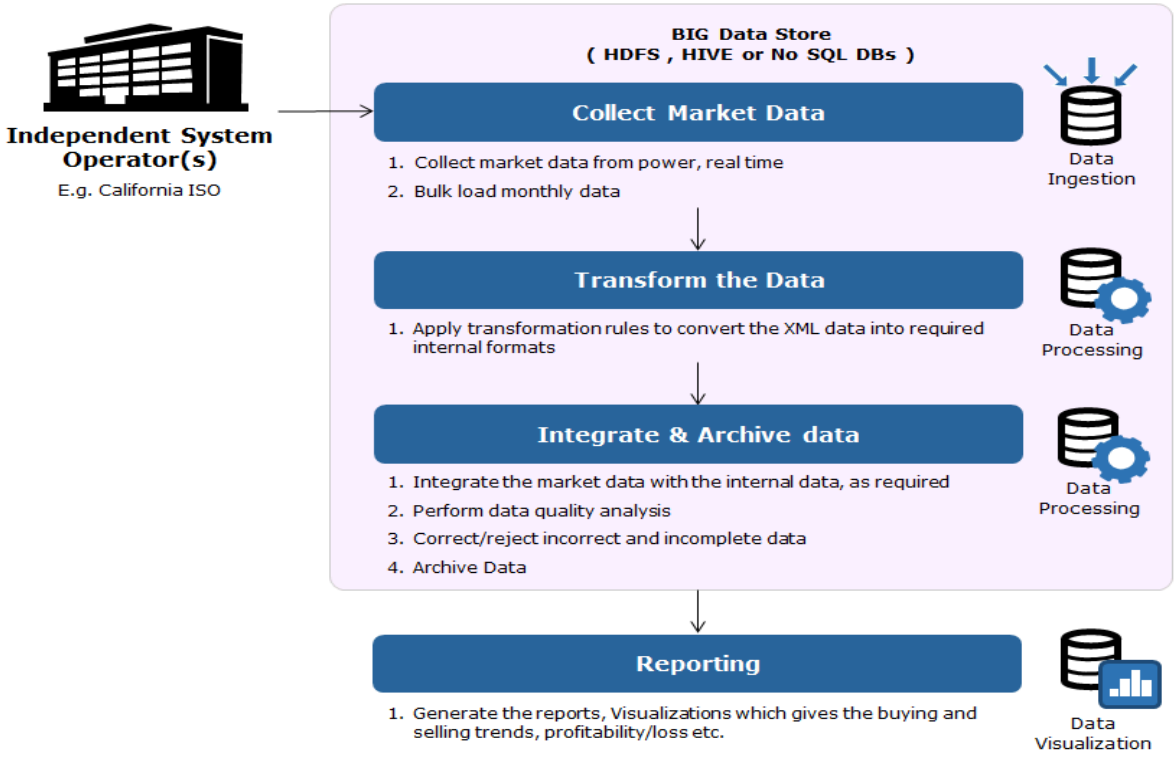
Data rows rejected because of data issues.
Cause: Data issue not handled thus the record was rejected.

USE CASES

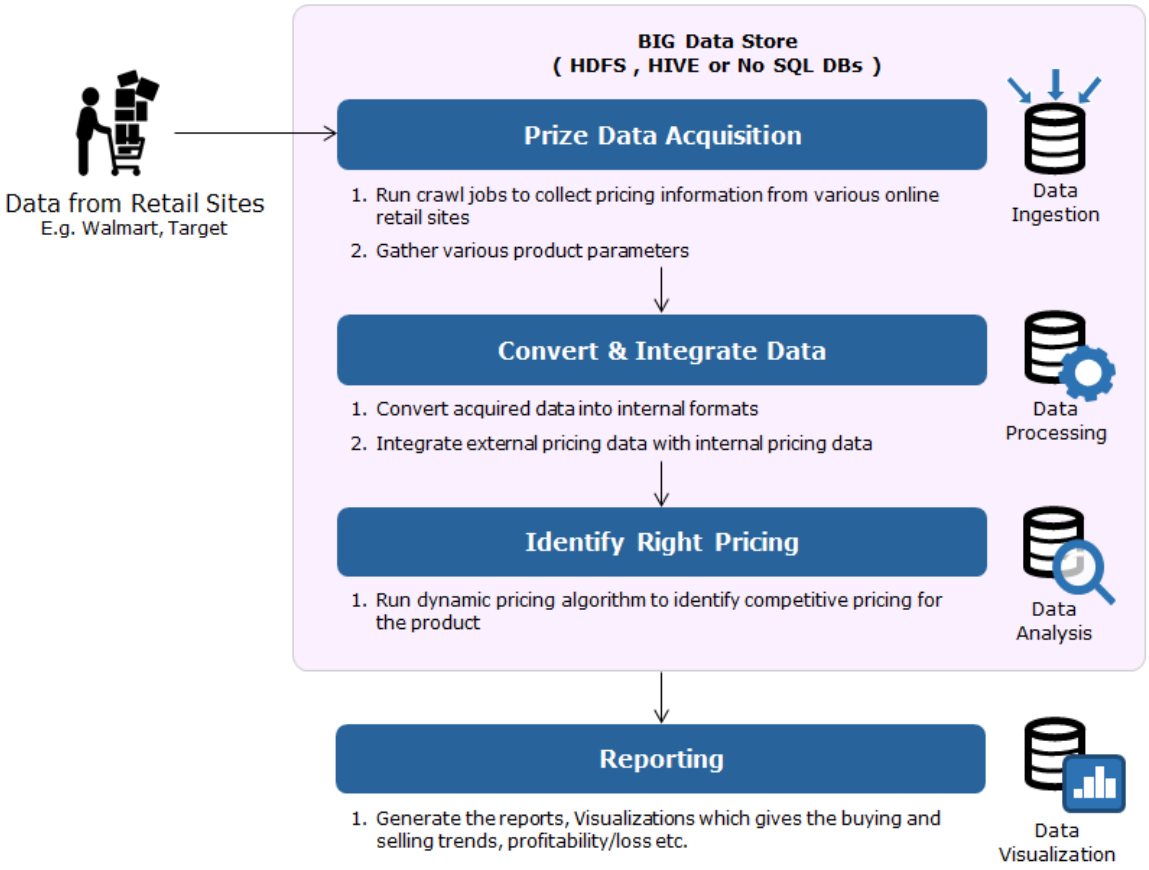
RETAIL - location based dynamic pricing



POWER TRANSMISSION - data acquisition



SALES - dynamic pricing



SKILLSET FOR BD TESTER

- NoSQL database knowledge
- HIVE query building knowledge
- Any scripting language for file handling
- Performance testing of batch jobs
- Failover testing skills
- ETL testing skills
- Unix skills
- Understanding of Hadoop ecosystem

CODES

Cleanse data and load files to Hadoop

```
// perform cleansing and save it
cat /root/Desktop/user_repo/To_Participant/AdditionalSuppliedFiles/patient_initial.txt | tr -s " "| tr " " ":" | tr -s ":" | sort -u | uniq > /root/Desktop/patient.txt
// view the file
cat /root/Desktop/patient.txt

// load from local system to Hadoop - loads in user directory
hdfs dfs -copyFromLocal /root/Desktop/patient.txt /user
// view in HDFS
hdfs dfs -cat /user/patient.txt
```

Hive table creation

```
hive
create table patient(pid INT,pname STRING,addr STRING,history STRING,dob DATE,allergies STRING) ROW FORMAT DELIMITED FIELDS TERMINATED BY ':'LINES TERMINATED BY '\n';
// load data from file on local system
LOAD DATA LOCAL INPATH '/root/Desktop/patient.txt' OVERWRITE INTO TABLE patient;
```

Data quality analysis

```
// count testing to verify number of rows in source and target
SELECT COUNT(*) FROM patient where year(dob)<=1995;
SELECT COUNT(*) FROM patient_target;

// sum testing
SELECT SUM(pid) FROM patient where year(dob)<=1995;
SELECT SUM(pid) FROM patient_target;

// match testing using intersect
select pid,pname,SUBSTR(addr,locate('+',addr)),history,dob,allergies from patient where year(dob)<=1995 intersect select * from patient_target;

// retrieve the details of patients born in the month of October
SELECT * from patient where month(dob) in (10);
```

COMMERCIAL HADOOP PLATFORMS

1. Cloudera: one of the earliest, has maximum installations running than any of its competitors. Incorporates Impala, which enables real-time massively parallel processing of BD to Hadoop.

2. Hortonworks: offers 100% open source Hadoop technology. It is also the first to integrate support for Apache HCatalog, which creates metadata, simplifying the process of sharing data across other services like Apache Hive or Pig.

3. Amazon Web Services: AWS is a hosted solution which integrates Hadoop with cloud-based data processing and storage services such as Amazon's Elastic Cloud Compute and Simple Storage Service (S3).

4. Microsoft HDInsight: It runs on Microsoft's Azure cloud platform and is the only big commercial Hadoop offering that runs in a Windows environment. It is based on Hortonworks.

5. Intel Distribution for Apache Hadoop: adds its Graph Builder and Analytics Toolkit functions to Hadoop. The company claims that its security updates to the Hadoop infrastructure offers added security to the data.

6. IBM: IBM's BigInsights package adds the company's proprietary analytics and visualization algorithms to the core Hadoop infrastructure.